

УНИВЕРСИТЕТ ИТМО

ТЕХНИЧЕСКИЙ ОТЧЁТ

**Агентная генерация вариантов использования и диаграмм
деятельности
по функциональным и нефункциональным требованиям**

Исполнитель: Боцман Артём

Версия отчёта: 13 сентября 2026 г.

Санкт-Петербург

2026

СОДЕРЖАНИЕ

РЕФЕРАТ	3
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
1 Постановка задачи	5
2 Обзор подходов и аналогов	6
3 Входные данные и типизированные модели	7
4 Архитектура и реализация	10
5 Методика экспериментальной оценки	12
6 Результаты экспериментов	15
7 Обсуждение результатов	20
8 Ограничения и угрозы достоверности	20
9 Воспроизводимость и развёртывание	21
ЗАКЛЮЧЕНИЕ	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
ПРИЛОЖЕНИЕ А. Карта файлов воспроизведения	23

РЕФЕРАТ

Отчёт содержит описание и экспериментальную проверку программной системы, которая преобразует функциональные и нефункциональные требования к приложению в структурированные варианты использования, пользовательские и системные истории, типизированные диаграммы деятельности, Mermaid-представления и двусторонний реестр трассировки. Система располагается после аналитического этапа и до кодогенерации: пользователь получает проверяемое описание поведения до запуска более дорогого этапа создания кода.

Решение реализовано на Python и LangGraph как два специализированных агентных подграфа и корневой оркестратор. Большая языковая модель интерпретирует смысл требований; детерминированный код назначает идентификаторы, проверяет схемы, целостность ссылок, графовые инварианты и ограничивает число исправлений. Итоговые изображения строятся Mermaid-рендерером по принятой типизированной модели; языковая модель не рисует изображение непосредственно.

На контролируемом наборе DEV20 выполнено по три независимых повтора одиночного вызова LLM и полного графа. Полный граф прошёл сквозную проверку в 60 из 60 запусков, одиночный вызов — в 2 из 60. Средняя по проектам F-мера трассировки составила 0,963 против 0,835. На отдельном наборе из 20 проектов руководителя размером от 6 до 74 функциональных требований полный граф завершил 19 из 20 проектов, одиночный вызов — 9 из 20; парная разность равна 0,50, 95%-й доверительный интервал [0,30; 0,70], $p = 0,001953$. Цена надёжности — увеличение медианного времени с 16,1 до 56,3 с и среднего расхода с 10,6 до 62,0 тыс. токенов на запуск DEV20.

Ключевые слова: требования, варианты использования, диаграммы деятельности, агентная система, LangGraph, трассировка, Mermaid, валидация, воспроизводимость.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Обозначение	Расшифровка
ФТ	функциональное требование
НФТ	нефункциональное требование
LLM	большая языковая модель
UC	вариант использования
E2E	сквозная успешность всего комплекта артефактов
F1	F-мера, гармоническое среднее точности и полноты
DEV20	открытая часть тестового набора из 20 проектов
Gold	зафиксированная эталонная разметка ожидаемых элементов
TraceManifest	машиночитаемый реестр прямых и обратных связей

1 Постановка задачи

1.1 Проблема и место решения в жизненном цикле

Исходные требования описывают функции и ограничения приложения естественным языком. Между ними и программным кодом отсутствует проверяемый поведенческий слой: границы процессов, участники, основной ход, альтернативы, ошибки и происхождение элементов часто остаются неявными. Если неоднозначность обнаруживается только после кодогенерации, исправление затрагивает уже созданный код, тесты и документацию.

Разработанная система располагается после аналитика и до кодирующего агента. Она уменьшает неоднозначность не за счёт общей переформулировки текста, а за счёт явного представления акторов, шагов, ожидаемых результатов, ветвлений и связей с исходными требованиями. Пользователь может проверить сценарии и диаграммы до кодогенерации и локализовать замечание до конкретного требования.

1.2 Почему недостаточно детерминированных средств

Pyreverse, GitDiagram, DeepWiki и другие средства обратного анализа получают структуру из существующего кода или репозитория. Они полезны после реализации, но не решают задачу до кода: не выводят из требований бизнес-цели, границы вариантов использования, альтернативные и ошибочные сценарии. Mermaid CLI и Structurizr DSL визуализируют уже заданное описание, но сами не интерпретируют требования.

LLM необходима для смысловой декомпозиции естественного языка. Однако вероятностный ответ без контроля может содержать пропуски, лишние элементы и некорректные ссылки. Поэтому применяется гибридный подход: LLM предлагает содержание, а типизированные модели, валидаторы и ограниченные циклы исправления обеспечивают контролируемость.

1.3 Цель и задачи

Цель работы — разработать и экспериментально проверить воспроизводимый агентный конвейер, который до кодогенерации формирует из ФТ и НФТ полный, машинно-проверяемый и трассируемый комплект вариантов использования и диаграмм деятельности.

- 1) определить типизированные модели входных требований, вариантов использования, сценарных шагов, диаграмм деятельности и связей трассировки;
- 2) реализовать два специализированных графа и корневой оркестратор с ограниченными циклами проверки и исправления;
- 3) реализовать формальные проверки, семантическую критику, ограниченное исправление и детерминированный рендеринг Mermaid;
- 4) подготовить тестовые наборы, эталонную разметку, валидаторы и метрики качества для входов разного размера;
- 5) сравнить полный граф с одиночным вызовом той же LLM по качеству, сквозной успешности, трассировке, времени и стоимости.

2 Обзор подходов и аналогов

Прямого количественного аналога с тем же входом и полным набором выходов не найдено. Поэтому аналоги разделены по фактическому назначению, а прямым экспериментальным базовым вариантом выбран одиночный вызов той же LLM с теми же входом, схемой результата и модулем оценки.

Таблица 1 — Сопоставление существующих подходов

Подход	Вход	Результат	Ограничение относительно задачи
Pyreverse	Python-код	структурные UML-диаграммы	код уже должен существовать; поведение из требований не выводится

Подход	Вход	Результат	Ограничение относительно задачи
GitDiagram	репозиторий	обзор структуры репозитория	работает после реализации и не формирует варианты использования
DeepWiki / DeepWiki-Open	репозиторий	документация и диаграммы	переносит LLM-вызовы в другой сервис; входом остаётся код
Mermaid CLI	готовый Mermaid-код	SVG/PNG	рендерер, а не средство анализа требований
Structurizr DSL	архитектурная модель	архитектурные представления	требует заранее заданной модели
Одиночный запрос к LLM	SpecificationReq	весь комплект за один ответ	нет промежуточной локализации ошибок и ограниченного восстановления
NOMAD	естественно-языковые требования	диаграммы классов	подтверждает принцип специализации агентов, но решает структурную, а не поведенческую задачу
Разработанное решение	SpecificationReq	UC, истории, Activity, Mermaid, трассировка, отчёт	более высокая цена; качество зависит от модели и эталонной разметки

Ferrari и соавторы показали, что при генерации UML из требований встречаются пропуски, неверные акторы, структурные дефекты и неполная трассировка. NOMAD демонстрирует полезность разделения задач между специализированными агентами, но отмечает неоднозначный эффект постоянной проверки. Query2Diagram использует принцип «типизированный граф — затем рендеринг» и отделяет структурные дефекты от смысловой релевантности. Эти наблюдения непосредственно повлияли на архитектуру и методику оценки данной работы.

3 Входные данные и типизированные модели

3.1 Входной контракт

Внешний контракт SpecificationReq содержит шесть полей: `project_task`, `project_name`, `project_goal`, `project_description`, `functional_requirements` и

non_functional_requirements. Первые четыре поля являются непустыми строками; ФТ и НФТ задаются списками строк. После приёма входа детерминированная функция назначает стабильные идентификаторы FR-001, FR-002, ... и NFR-001, NFR-002, Явные перечисления и части, разделённые точкой с запятой, могут быть атомизированы в FRA-...; союзы автоматически не расщепляются, чтобы не изменить бизнес-смысл.

Таблица 2 — Основные типизированные сущности

Сущность	Ключевые поля	Назначение
SpecificationRequest	задача, название, цель, описание, ФТ, НФТ	нормализованный вход со стабильными ID
UseCase	актор, цель, триггер, предусловия, сценарии, постусловия	граница бизнес-процесса
ScenarioStep	ID, порядок, actor_id, действие, ожидаемый результат, source_fr_ids	атомарный проверяемый шаг
ActivityDiagram	дорожки, вершины, рёбра, условия, use_case_id	типизированная модель поведения
TraceLink	источник, назначение, тип связи, происхождение	явная связь между уровнями
ValidationReport	статус и список блокирующих/неблокирующих проблем	решение о принятии или исправлении

3.2 Структура варианта использования

Вариант использования хранится не как произвольный абзац. Обязательными являются имя, цель, основной актер, триггер, основной сценарий и хотя бы одна ссылка на ФТ. Дополнительно задаются вторичные акторы, предусловия, успешные и неуспешные постусловия, альтернативные и исключительные сценарии, пользовательские и системные истории, отсутствующая информация и неподтверждённые предположения.

Каждый сценарный шаг содержит исполнителя, действие, ожидаемый результат и ссылки на исходные ФТ. Пользовательская история формулирует ценность для роли, а системная история — внутреннюю ответственность

системы. Если `actor_id` отсутствует, действие относится к самой проектируемой системе, а не к оркестратору или LLM.

3.3 Структура диаграммы деятельности

Для каждого принятого варианта использования создаётся одна диаграмма деятельности. Она содержит дорожки участников, вершины начала, действия, решения, объединения и завершения, а также ориентированные рёбра с условиями. Поле `use_case_id` связывает диаграмму с вариантом использования; `related_step_ids` у вершины или ребра указывает конкретные шаги сценария, из которых элемент получен. Система не генерирует множество картинок для выбора лучшей: одна типизированная модель последовательно проверяется и при необходимости исправляется.

4 Архитектура и реализация

Общая архитектура решения

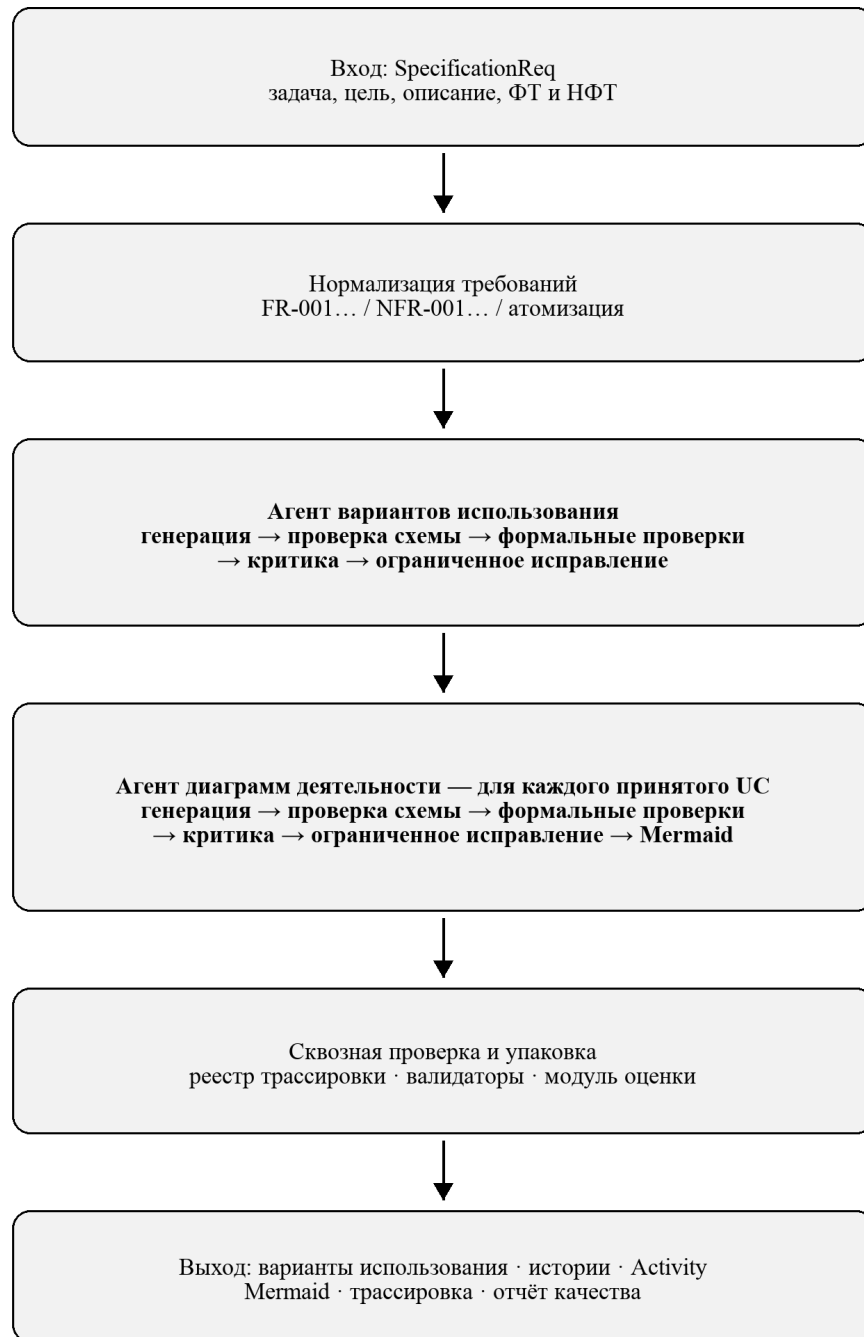


Рисунок 1 — Общая архитектура агентного конвейера

4.1 Состав графов

Исполняемых содержательных агентов два: агент вариантов использования и агент диаграмм деятельности. Корневой граф является

оркестратором: он нормализует вход, запускает первый подграф, последовательно запускает второй подграф для каждого принятого УС, выполняет сквозную проверку, модуль оценки и упаковку результата. Генератор, критик и редактор являются внутренними LLM-ролями, а не отдельными агентами.

Таблица 3 — Статический состав графов

Граф	Рабочие вершины	Логические переходы	Функция
Корневой оркестратор	6	7	порядок, состояние, цикл по УС, итоговая проверка
Агент вариантов использования	9	13	генерация, схема, формальные проверки, критика, решение, исправление
Агент диаграмм деятельности	10	14	генерация Activity, проверки, критика, исправление, Mermaid
Итого	25	34	три связанных графа

4.2 Проверка, критика и ограниченное исправление

Проверка схемы гарантирует типы, обязательные поля и форматы идентификаторов. Детерминированные валидаторы проверяют покрытие требований, уникальность, ссылочную целостность, начало и конец диаграммы, достижимость, допустимые переходы, ветвления и принадлежность элементов существующим шагам. При блокирующей формальной ошибке дорогой критик может быть пропущен: системе уже известен точный код дефекта и затронутые ID.

Семантический критик получает типизированный результат и отчёты валидаторов, оценивает содержательные пропуски и возвращает структурированный вердикт. Редактор получает не общий запрос «сделать лучше», а список проблем и ID элементов. По умолчанию разрешено не более двух исправлений. Ограничение предотвращает бесконечный цикл, фиксирует верхнюю границу стоимости и переводит неустранимый дефект в

явный контролируемый отказ. Значение настраиваемо от 0 до 10; два повтора выбраны как инженерный компромисс и отдельно исследуются.

4.3 Трассировка и рендеринг

TraceManifest хранит прямые и обратные связи по цепочке ФТ → вариант использования → сценарный шаг → вершина или ребро диаграммы. Ссылка на несуществующий конечный элемент не является допустимой: материализация и валидаторы должны либо создать согласованный элемент, либо удалить/отклонить висячую ссылку. Благодаря этому по замечанию к диаграмме можно найти исходное требование, а по требованию — все связанные сценарии и элементы.

После принятия Activity-модели обычная функция создаёт Mermaid-текст. Строка flowchart TD задаёт направленный сверху вниз граф; затем объявляются узлы, дорожки, переходы и подписи условий. Mermaid CLI преобразует текст в SVG/PNG. Рендерер не принимает решений о бизнес-смысле и не заменяет валидаторы.

4.4 Хранение и обработка ошибок

Промежуточное состояние может сохраняться в SQLite с thread_id. Повторные запросы к провайдеру ограничены по числу вызовов, токенам и оценочной стоимости; для временных кодов 408, 409, 425, 429 и 5xx применяется повтор с задержкой. Секреты читаются из локальных .env-файлов, которые исключены из Git. В публичный комплект записываются безопасная конфигурация, обезличенная телеметрия и итоговые артефакты.

5 Методика экспериментальной оценки

5.1 Тестовые наборы

Таблица 4 — Использованные тестовые наборы

Набор	Состав	Назначение	Статус эталона
Контролируемый benchmark	30 проектов: DEV20 и sealed hidden10; 8 простых, 13 средних, 9 сложных; 13 RU и 17 EN	качество содержания, трассировка, повторяемость	авторская техническая Gold-разметка; hidden не использован
Набор руководителя	20 проектов, четыре группы по 5; 6–74 ФТ	масштабирование, E2E, время, стоимость	Gold-кандидат заморожен, ожидает двух независимых экспертов

Контролируемые проекты синтетически сформированы из типовых бизнес-процессов. Для каждого заранее заданы ожидаемые акторы, варианты использования, ключевые этапы, ветвления, связи ФТ → УС, допустимые синонимы и запрещённые предположения. Набор руководителя получен отдельно и не является продолжением этих 30 кейсов; для него подготовлена собственная Gold-разметка-кандидат.

5.2 Сравнимые режимы

Основное сравнение проводится между одиночным вызовом LLM и полным графом. Одинаковы вход, версия модели, провайдер, целевая JSON Schema и модуль оценки; различается только организация процесса. Дополнительно исследованы режимы без критика и без исправления, а также переносимость на DeepSeek, GPT-5.5 и Claude Opus 5.

5.3 Метрики

Для сопоставления с Gold используются точность $P = TP/(TP+FP)$, полнота $R = TP/(TP+FN)$ и $F1 = 2PR/(P+R)$. TP — подтверждённый элемент, FP — лишний неподтверждённый элемент, FN — пропущенный ожидаемый элемент. Эквивалентные названия учитываются по заранее зафиксированным правилам, а не после просмотра результата.

Таблица 5 — Метрики и их практический смысл

Метрика	Объект	Интерпретация
Actor F1	акторы	правильно ли определены участники процесса
UC F1	варианты использования	правильно ли выделены границы процессов
Milestone F1	основные шаги	сохранены ли обязательные этапы
Branch F1	альтернативные/ошибочные пути	сохранены ли бизнес-условия и исключения
Trace F1	связи ФТ → UC	можно ли доказать происхождение результата
FR/Activity coverage	требования и элементы Activity	ничего ли не потеряно между уровнями
Activity structural validity	структура диаграммы	пройденны ли детерминированные инварианты
E2E success	весь комплект	можно ли принять результат без ручного ремонта формата и ссылок
Время, токены, стоимость	журнал LLM-вызовов	цена получения принимаемого результата

5.4 Статистический протокол

На DEV20 выполнено по три независимых повтора на проект и режим. Для средних значений по проектам использован кластерный bootstrap с 95%-м доверительным интервалом; единицей анализа является проект, а повторы усредняются внутри проекта. Для парной бинарной E2E-метрики применён точный двусторонний критерий Мак-Немара/биномиальный критерий, множественные проверки DEV20 скорректированы методом Холма. На размерном наборе выполнен один запуск на проект, поэтому доверительный интервал отражает различия между проектами, но не стохастическую повторяемость.

6 Результаты экспериментов

6.1 Основное сравнение на DEV20

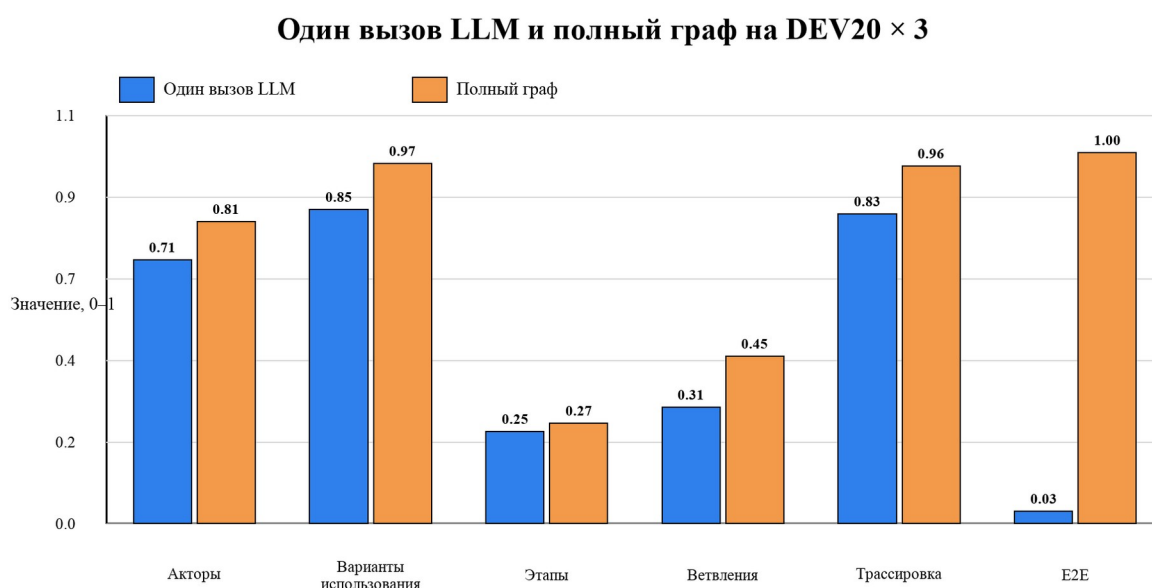


Рисунок 2 — Сравнение одиночного вызова и полного графа на DEV20 × 3

Таблица 6 — Макросредние значения по проектам и 95%-е доверительные интервалы

Метрика	Один вызов LLM	Полный граф	Разность
Actor F1	0,710 [0,607; 0,810]	0,814 [0,744; 0,880]	+0,104
UC F1	0,847 [0,747; 0,933]	0,970 [0,922; 1,000]	+0,124
Milestone F1	0,249 [0,199; 0,297]	0,271 [0,214; 0,324]	+0,021
Branch F1	0,314 [0,188; 0,456]	0,452 [0,304; 0,606]	+0,137
Trace F1	0,835 [0,733; 0,925]	0,963 [0,929; 0,990]	+0,128
E2E success	2/60 (0,033)	60/60 (1,000)	+0,967

Наиболее сильный подтверждённый результат — рост сквозной принимаемости с 2/60 до 60/60: +96,7 процентного пункта, 95%-й ДИ [+91,7; +100,0], $p = 5,72 \times 10^{-6}$ после поправки Холма. Прирост Milestone F1 мал, поэтому утверждать универсальное улучшение каждой содержательной характеристики нельзя. Подтверждённый вывод состоит в повышении формальной надёжности, границ процессов и трассировки.

6.2 Связи требований с вариантами использования

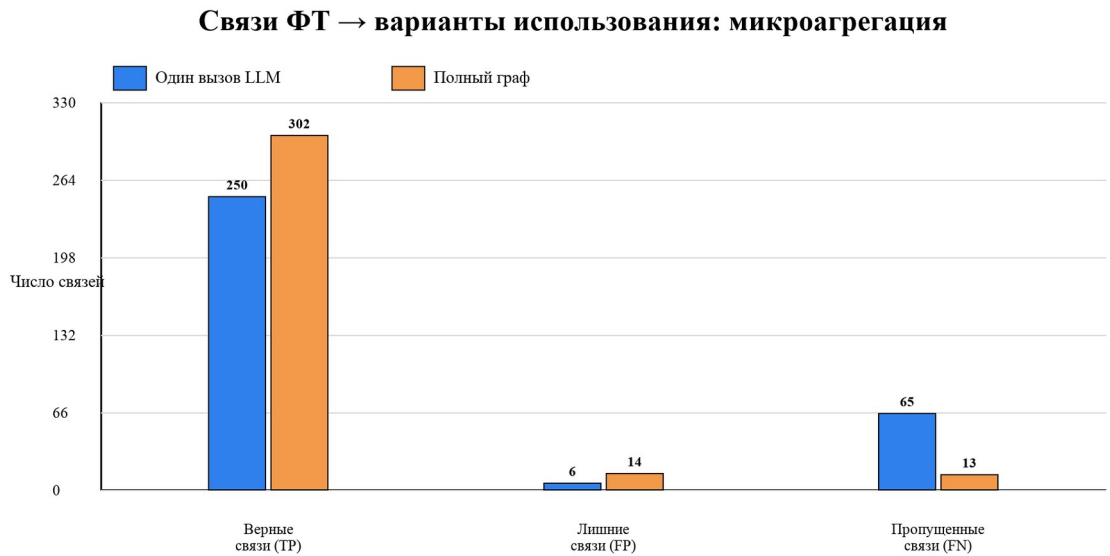


Рисунок 3 — Верные, лишние и пропущенные связи ФТ → УС

Таблица 7 — Микроагрегация связей ФТ → УС на DEV20

Режим	Ожидалось	TP	FP	FN	Precision	Recall	F1
Один вызов	315	250	6	65	0,977	0,794	0,876
Полный граф	315	302	14	13	0,956	0,959	0,957

Полный граф нашёл на 52 корректные связи больше и сократил пропуски с 65 до 13. Одновременно число лишних связей увеличилось с 6 до 14, поэтому Precision немного снизилась. Итоговая микро-F1 выросла с 0,876 до 0,957. Макро Trace F1 в таблице 6 отличается, поскольку сначала вычисляется по каждому проекту, а затем усредняется; обе агрегации показывают один и тот же механизм — значительное уменьшение потерянных связей.

6.3 Операционные затраты

Таблица 8 — Время, токены и оценочная стоимость на DEV20

Режим	Медиана времени	p95 времени	Средние токены	Средние вызовы	Средняя стоимость
Один вызов	16,1 с	20,7 с	10 565	1,00	\$0,0069
Полный граф	56,3 с	90,6 с	62 046	11,82	\$0,0346

Полный граф требует примерно в 3,5 раза больше медианного времени и в 5,9 раза больше токенов. Стоимость является расчётной по сохранённой телеметрии и тарифам провайдера, а не значением банковского счёта. Дополнительный расход оправдан только там, где нужен автоматически принимаемый и объяснимый комплект.

6.4 Масштабирование на проектах руководителя

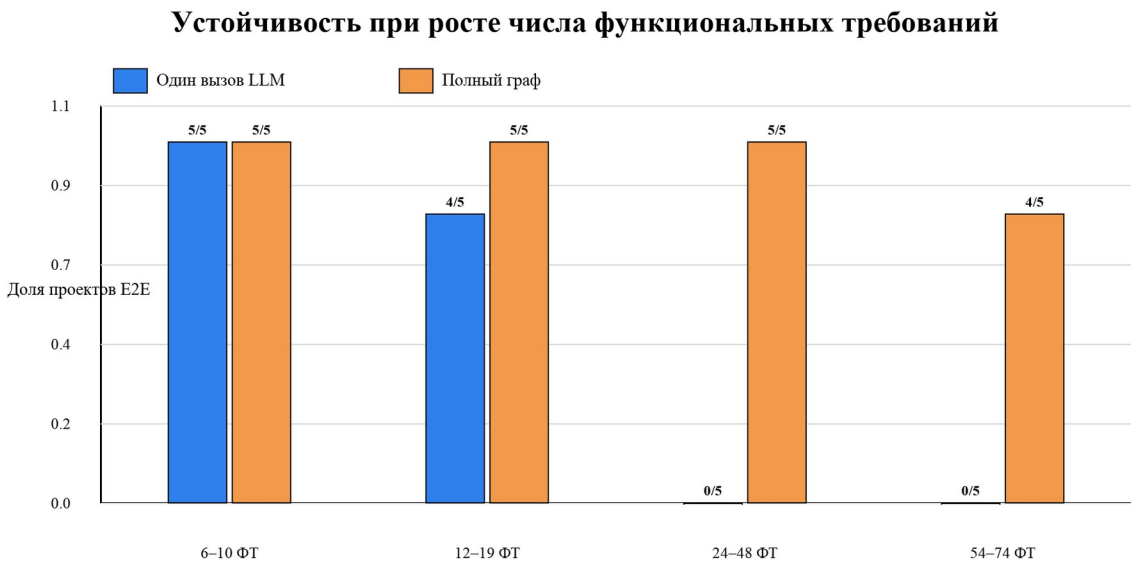


Рисунок 4 — Сквозная успешность в четырёх группах размера

Таблица 9 — E2E-успех по числу функциональных требований

Группа	Проектов	Один вызов	Полный граф
6–10 ФТ	5	5/5	5/5
12–19 ФТ	5	4/5	5/5
24–48 ФТ	5	0/5	5/5
54–74 ФТ	5	0/5	4/5

Группа	Проектов	Один вызов	Полный граф
Итого	20	9/20	19/20

Парная разность E2E составляет +0,50; 95%-й ДИ [+0,30; +0,70], точное двустороннее $p = 0,001953$. У одиночного вызова успешность исчезает начиная с группы 24–48 ФТ, тогда как полный граф сохраняет 19/20 принимаемых комплектов. Единственный отказ FULL на SCALE-018 является контролируемым: после двух исправлений одна из 44 диаграмм сохранила неподтверждённую ветвь и структурный дефект решения, поэтому итоговый gate отклонил комплект.

6.5 Вклад критика

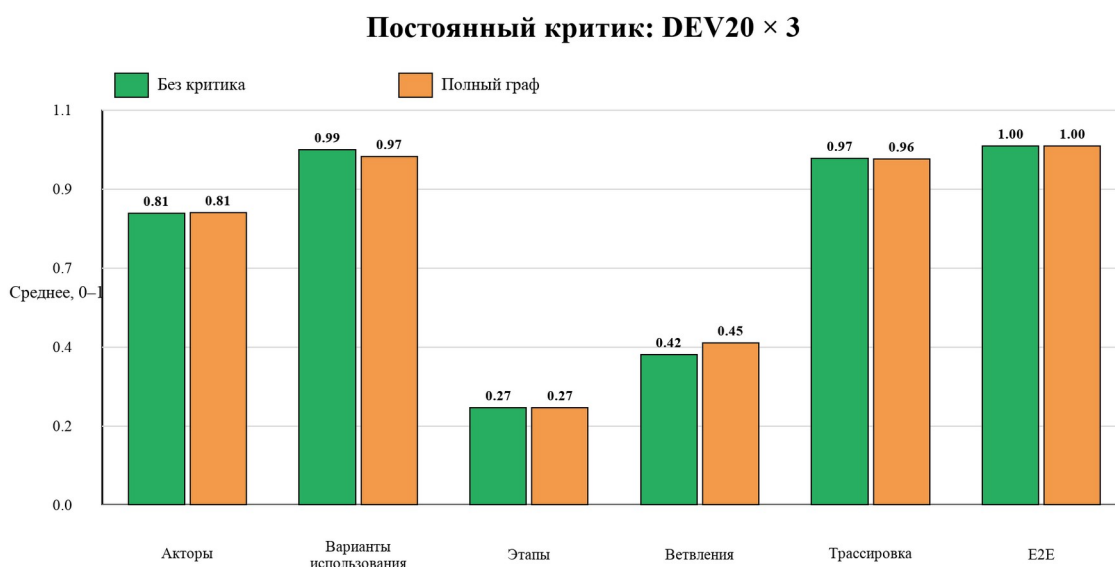


Рисунок 5 — Сравнение полного графа и режима без постоянного критика

Оба режима завершили 60/60 запусков. Наибольшая наблюдаемая разность — Branch F1 +0,034, но 95%-й ДИ [−0,015; +0,088] включает ноль. Среднее число вызовов составило 11,82 для FULL и 7,82 без критика; токены — 62 046 и 47 098. Следовательно, постоянный критик на DEV20 не доказал отдельного прироста и должен запускаться условно: при сложном случае или содержательном сомнении после формальных проверок. Вывод

предварителен, поскольку сохранённые режимы получены на разных программных ревизиях организации контрольных точек.

6.6 Переносимость между моделями

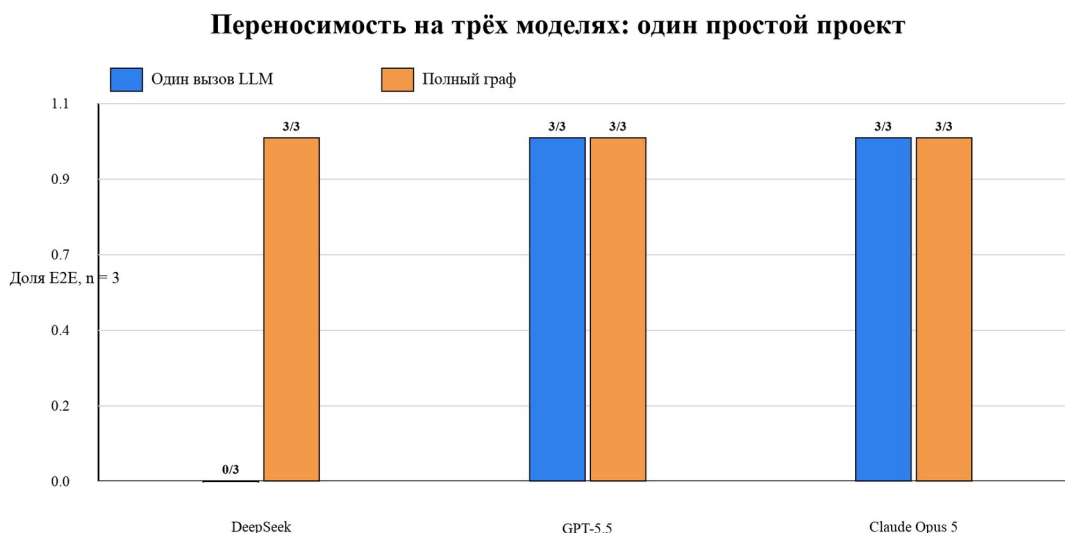


Рисунок 6 — Диагностический пилот на одном простом проекте, три повтора

DeepSeek выбран как доступная модель для массовых повторов, GPT-5.5 и Claude Opus 5 — как сильные модели двух независимых провайдеров. Такой выбор отделяет вклад архитектуры от эффекта одной модели. На B1-DEV-002 FULL изменил E2E DeepSeek с 0/3 на 3/3; у GPT-5.5 и Claude оба режима дали 3/3. У GPT-5.5 Milestone F1 был выше у одиночного вызова, у Claude — у полного графа. Это один простой кейс, поэтому результат подтверждает совместимость, но не ранжирует модели.

6.7 Проверка программной реализации

Автоматический набор содержит 87 тестов; все тесты прошли 16 сентября 2026 г. Отдельный mutation-suite обнаружил 15 из 15 заранее подготовленных классов структурных и трассировочных дефектов. Это означает 100%-е обнаружение только для заявленного набора мутаций и не является доказательством обнаружения любого возможного смыслового дефекта.

7 Обсуждение результатов

Практический результат работы — не просто изображение диаграммы. Кодировущий агент и пользователь получают типизированные процессы, явные входные требования, альтернативы и проверяемые связи. Пользователь может согласовать более дешёвые сценарии и диаграммы до генерации кода; кодирующий и тестирующий агенты получают основу для декомпозиции и проверок.

Эксперименты не подтверждают тезис «граф всегда умнее одного запроса». На простом входе сильная модель может сразу выдать корректный результат, а постоянная критика может не окупаться. Подтверждён более точный тезис: при одинаковой модели полный граф значительно повышает вероятность получить целый, структурно допустимый и трассируемый комплект, особенно при росте входа.

Низкие Milestone и Branch F1 показывают оставшуюся исследовательскую проблему: Gold и модель могут по-разному выбирать уровень детализации и границы ветвлений. Повышать итоговую оценку только за счёт промпта нельзя; требуется независимая экспертная проверка допустимых эквивалентов и отдельный анализ дробления шагов.

8 Ограничения и угрозы достоверности

- Gold-разметка DEV20 является авторской технической разметкой; Gold-кандидат размерного набора ожидает проверки двумя независимыми экспертами.
- Hidden10 не использовался для настройки и не вскрывался; внешняя валидность на нём пока не заявляется.
- На размерном наборе выполнен один запуск на проект; нужен минимум трёхкратный повтор после экспертного freeze.
- Содержательные метрики зависят от правил эквивалентности, уровня декомпозиции и качества Gold.

- Сравнение трёх моделей выполнено на одном простом проекте и служит пилотом переносимости, а не рейтингом моделей.
- Сравнение критика использует результаты разных программных ревизий; для причинного вывода нужен повторный контролируемый запуск одной ревизии.
- Расчётная стоимость зависит от тарифов и кэширования провайдера; её следует пересчитывать по сохранённым токенам.
- Pyreverse, GitDiagram и DeepWiki не являются прямыми количественными baseline, поскольку начинают с кода или репозитория.

9 Воспроизводимость и развёртывание

Репозиторий содержит фиксированные зависимости, MIT License, README, офлайн-проверку GitHub Actions, единый интерфейс командной строки, тестовые наборы, конфигурации и сохранённые результаты. API-ключи не входят в репозиторий. Базовый запуск выполняется в изолированном окружении; платный режим требует явного разрешения `--allow-live`.

Таблица 10 — Основные команды воспроизведения

Действие	Команда
Установка	<code>poetry install</code>
Проверка тестов	<code>poetry run pytest -q</code>
Одиночный и полный эксперимент	<code>poetry run python scripts/run_benchmark_experiment.py --help</code>
Проверка сохранённого эксперимента	<code>poetry run python scripts/verify_saved_experiment.py --help</code>
Построение итоговых таблиц и рисунков	<code>poetry run python scripts/build_final_size_and_critic_evidence.py</code>

Открытый репозиторий:
https://github.com/ArtemBotsman/agent_langgraph_diagrams. Публичный
 репозиторий не должен содержать `.env`, ключи, абсолютные локальные пути,

закрытую Gold-разметку hidden-набора или необезличенные ответы провайдеров.

ЗАКЛЮЧЕНИЕ

Разработан полный технологический контур от SpecificationReq до вариантов использования, историй, диаграмм деятельности, Mermaid, трассировки и отчёта качества. Реализованы два специализированных LangGraph-агента, корневой оркестратор, типизированные контракты, детерминированные валидаторы, семантическая критика, ограниченное исправление, сохранение состояния и воспроизводимый экспериментальный контур.

На DEV20 × 3 полный граф достиг 60/60 E2E против 2/60 у одиночного вызова и повысил макро Trace F1 с 0,835 до 0,963. На 20 проектах руководителя размером 6–74 ФТ полный граф завершил 19/20 против 9/20, что даёт статистически значимую парную разность +0,50. Эти данные подтверждают технологическую ценность графа как надёжного проверяемого этапа до кодогенерации, но не доказывают универсальное смысловое превосходство на любых моделях.

Дальнейшая работа: независимая проверка Gold двумя экспертами и оценка согласия; трёхкратные повторы на размерном наборе; однократный sealed hidden-run после научного freeze; исследование причин снижения Milestone/Branch F1 у GPT-5.5; сравнение лимитов 0, 1, 2, 3 и 4 исправления; условный запуск критика после формальной проверки; проверка ещё более крупных входов и пакетное сохранение после каждой диаграммы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ferrari A., Abualhaija S., Arora C. Model Generation with LLMs: From Requirements to UML Sequence Diagrams // Proceedings of the 32nd IEEE International Requirements Engineering Conference Workshops. 2024. С. 291–300. DOI: 10.1109/REW61692.2024.00044.

2. Giannouris P., Ananiadou S. NOMAD: A Multi-Agent LLM System for UML Class Diagram Generation from Natural Language Requirements. arXiv:2511.22409. 2025. URL: <https://arxiv.org/abs/2511.22409>.
3. Baryshnikov O., Alekseev A. M., Nikolenko S. I. Query2Diagram: Answering Developer Queries with UML Diagrams. arXiv:2604.23816. 2026. URL: <https://arxiv.org/abs/2604.23816>.
4. Zhang W., Jiang B., Fu Y. et al. Large language models in model-driven engineering: a systematic mapping study // Empirical Software Engineering. Опубликовано 16.07.2026. DOI: 10.1007/s10664-026-10921-4.
5. Object Management Group. Unified Modeling Language, Version 2.5.1. 2017. URL: <https://www.omg.org/spec/UML/2.5.1/>.
6. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering.
7. LangChain. LangGraph documentation. URL: <https://docs.langchain.com/oss/python/langgraph/overview>.
8. Mermaid. Flowchart syntax. URL: <https://mermaid.js.org/syntax/flowchart.html>.
9. Pylint. Pyreverse documentation. URL: https://pylint.readthedocs.io/en/latest/additional_tools/pyreverse.html.
10. GitDiagram: repository-to-diagram tool. URL: <https://github.com/ahmedkhaleel2004/gitdiagram>.
11. DeepWiki Open: repository-to-wiki and diagram system. URL: <https://github.com/AsyncFuncAI/deepwiki-open>.
12. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления.

ПРИЛОЖЕНИЕ А. Карта файлов воспроизведения

Назначение	Путь в репозитории
Контролируемый набор 30 проектов	benchmark/v1_0_synthetic/
Набор руководителя 20 проектов	benchmark/size_scaling_v1/

Назначение	Путь в репозитории
Результаты DEV20 × 3	artifacts/benchmark_runs/b1-dev20-r3-deepseek-flash-2026-09-13/ и full-dev20-r3-deepseek-flash-2026-09-13/
Итоговая статистика DEV20	artifacts/final_dev20_statistical_evidence_2026-09-13/
Масштабирование и критик	artifacts/final_experiment_evidence_2026-09-16/
Три модели	artifacts/final_experiment_suite_2026-09-16/
Основной запуск	scripts/run_benchmark_experiment.py
Оценка внешней модели	scripts/evaluate_external_model_outputs.py
Сборка данного отчёта	scripts/build_final_report_gost_2026_09_16.py

Контрольная дата приведённых результатов — 16 сентября 2026 г. Сохранённые исходные ответы и диагностические отказы не удалялись; повторные платные вызовы для подготовки отчёта не выполнялись.